Experiment No. 5

AIM:

To automate application deployment using Github/ Jenkins.

THEORY:

Continuous Integration and Continuous Deployment (CI/CD) form the backbone of modern DevOps methodologies, aiming to bridge the gap between development teams and operational deployment environments. Continuous Integration (CI) focuses on frequently merging developer code changes into a central repository, followed by automated builds and tests to detect integration bugs early. Continuous Deployment (CD) extends this pipeline by automatically releasing verified code changes to production or staging environments, minimizing human error and accelerating the release cycle.

At the core of this workflow is Jenkins, an open-source automation server that orchestrates software development workflows through pipelines. A Jenkins Pipeline is a suite of plugins that supports implementing and integrating continuous delivery pipelines into Jenkins using code. Written as a Jenkinsfile, these pipelines utilize a domain-specific language (DSL) based on Groovy. Declarative Pipelines provide a structured, simplified syntax consisting of predefined structural elements: pipeline (the root container), agent (which defines the execution machine or node), stages (a sequence of tasks), and steps (the individual commands executed within a stage).

The end-to-end automation model relies on continuous feedback mechanisms between version control systems and the automation server. Using Git for Version Control System (VCS) management alongside GitHub as the remote repository, developers push source code and pipeline definition files directly to remote branches. Automation is triggered either through polling (where Jenkins periodically checks GitHub for new commits) or, more efficiently, via GitHub Webhooks. A webhook sends an immediate HTTP POST payload from GitHub to Jenkins upon every git push event, prompting Jenkins to fetch the latest code commit, parse the Jenkinsfile, and execute the defined Build, Test, and Deploy phases in isolated stages.

Each stage in the pipeline serves a specific purpose in validating code quality:

- Build Stage: Compiles source code resolves external dependencies, or packages assets into executable units (e.g., binaries, JAR files, or web bundles).
- Test Stage: Runs unit tests, integration tests, or static code analysis to ensure code modifications do not introduce regressions or security vulnerabilities.
- Deploy Stage: Transfers tested application artifacts directly to web servers, cloud containers, or local hosting directories, ensuring immediate availability to end-users.
- Pipeline as Code (Jenkinsfile): Storing the Jenkinsfile directly in your GitHub repository alongside your project code ensures your CI/CD configuration is version-controlled, auditable, and easily recoverable.

- Environment Isolation & Agents: While agent any runs jobs on the local host machine, production setups typically use dedicated build agents (slaves) or Docker containers to isolate dependencies and prevent build conflicts.
- Automated Rollbacks: Advanced pipeline deployment stages are often designed with conditional blocks (post { failure { ... } }) to automatically revert to a previous working deployment if tests or deployment steps fail.
- Secrets Management: Sensitive data like API keys, SSH credentials, and GitHub tokens should never be hardcoded in the Jenkinsfile; instead, manage them securely using the Jenkins Credentials Manager.
- Pipeline Visibility & Artifact Tracking: Jenkins provides visual stage views to monitor build durations and progress, along with post-build steps like archiveArtifacts to store compiled outputs for future releases.

IMPLEMENTATION:

```
C:\Users\yogini bhatia>java -version
openjdk version "21.0.12" 2026-07-21 LTS
OpenJDK Runtime Environment Temurin-21.0.12+8 (build 21.0.12+8-L
TS)
OpenJDK 64-Bit Server VM Temurin-21.0.12+8 (build 21.0.12+8-LTS,
 mixed mode, sharing)
```

```
Microsoft Windows [Version 10.0.26200.8894]
(c) Microsoft Corporation. All rights reserved.

C:\Users\yogini bhatia>java -version
openjdk version "21.0.12" 2026-07-21 LTS
OpenJDK Runtime Environment Temurin-21.0.12+8 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Temurin-21.0.12+8 (build 21.0.12+8-LTS, mixed mode, sharing)

C:\Users\yogini bhatia>mkdir jenkins-html-demo

C:\Users\yogini bhatia>cd jenkins-html-demo

C:\Users\yogini bhatia\jenkins-html-demo>echo "<html><body><h1>Hello Jenkins CI/CD!</h1></body></html>" > index.html

C:\Users\yogini bhatia\jenkins-html-demo>echo pipeline { agent any; stages { stage('Build') { steps { echo 'Building the application.
```

```
C:\Users\yogini bhatia\jenkins-html-demo>echo "<html><body><h1>Hello Jenkins CI/CD!</h1></body></html>" > index.html

C:\Users\yogini bhatia\jenkins-html-demo>echo pipeline { agent any; stages { stage('Build') { steps { echo 'Building the application.
..' } } stage('Test') { steps { echo 'Testing the application...' } } stage('Deploy') { steps { echo 'Deploying the application...' }
 } } } > Jenkinsfile

C:\Users\yogini bhatia\jenkins-html-demo>git init
Initialized empty Git repository in C:/Users/yogini bhatia/jenkins-html-demo/.git/

C:\Users\yogini bhatia\jenkins-html-demo>git add .

C:\Users\yogini bhatia\jenkins-html-demo>git commit -m "Initial commit with app and Jenkinsfile"
[master (root-commit) a448996] Initial commit with app and Jenkinsfile
 2 files changed, 2 insertions(+)
 create mode 100644 Jenkinsfile
 create mode 100644 index.html

C:\Users\yogini bhatia\jenkins-html-demo>git branch -M main

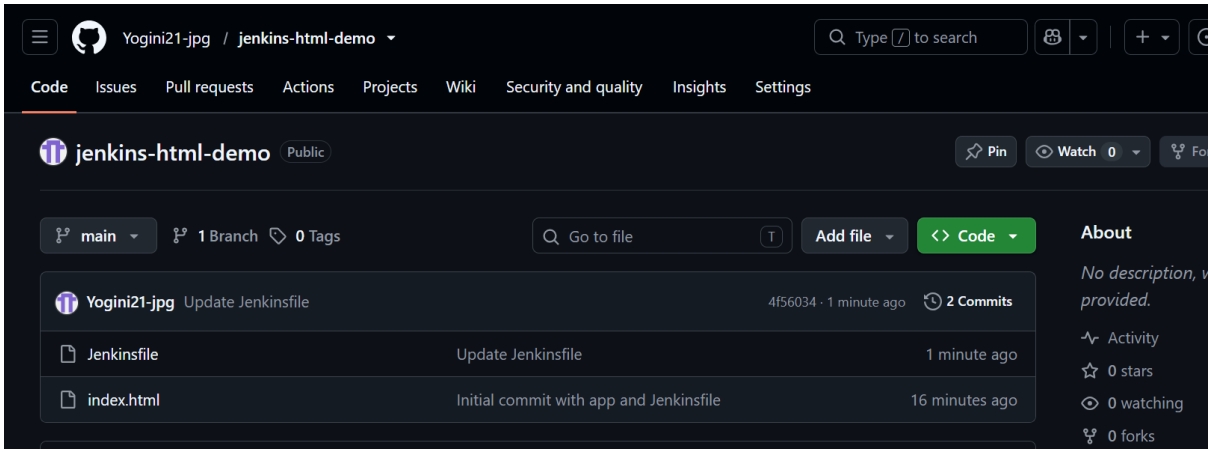
C:\Users\yogini bhatia\jenkins-html-demo>git remote add origin https://github.com/Yogini21-jpg/jenkins-html-demo.git

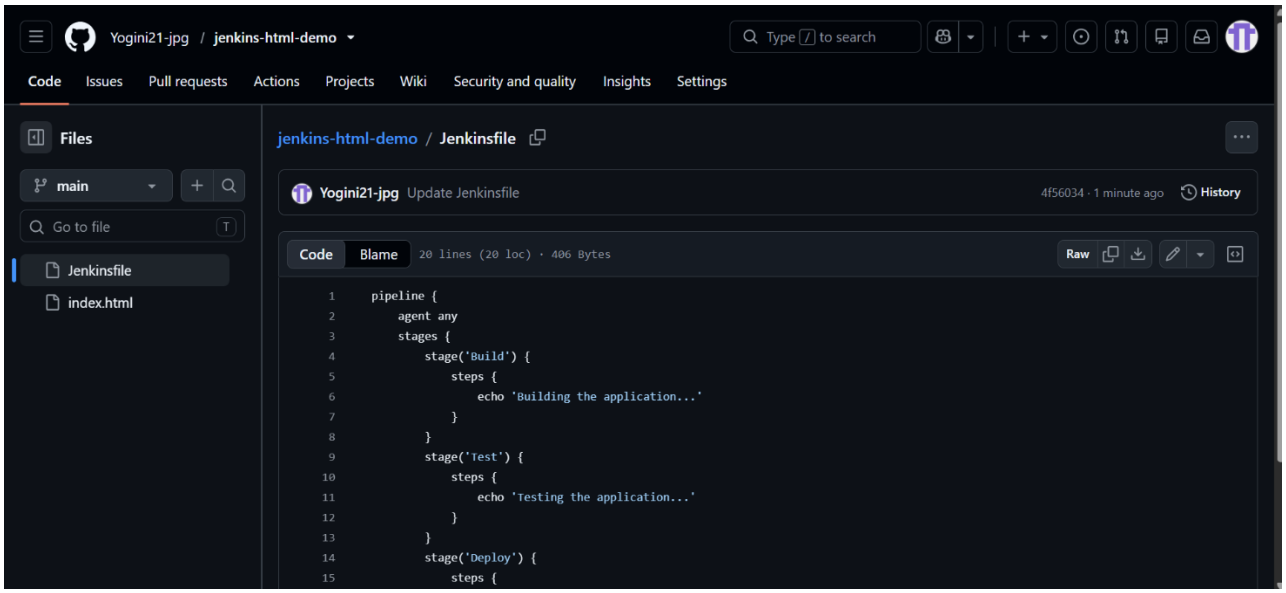
C:\Users\yogini bhatia\jenkins-html-demo>git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 440 bytes | 440.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Yogini21-jpg/jenkins-html-demo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

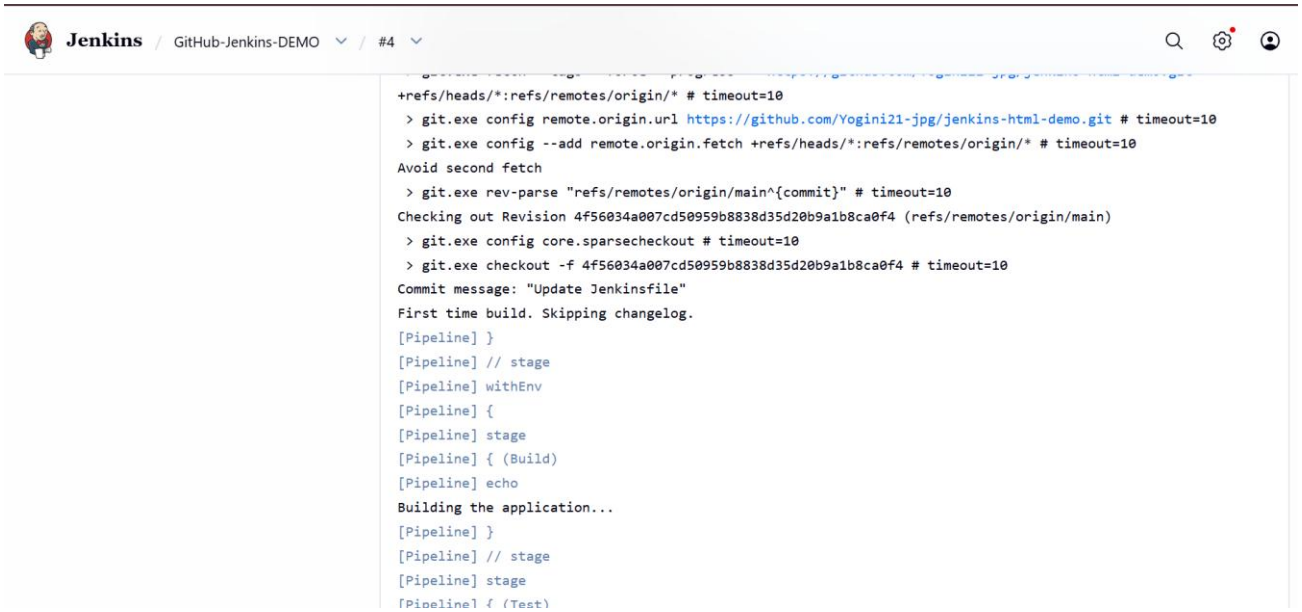


GitHub-Jenkins-DEMO / #1

```
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Testing the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```


Yogini21-jpg / jenkins-html-demo
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
jenkins-html-demo
Public
Pin
Watch 0
main
1 Branch
0 Tags
Go to file
Add file
Code
About
Yogini21-jpg Update Jenkinsfile
4f56034 · 1 minute ago
2 Commits
Jenkinsfile
Update Jenkinsfile
1 minute ago
index.html
Initial commit with app and Jenkinsfile
16 minutes ago
Activity
0 stars
0 watching
0 forks


Yogini21-jpg / jenkins-html-demo
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
Files
main
Go to file
Jenkinsfile
index.html
jenkins-html-demo / Jenkinsfile
Yogini21-jpg Update Jenkinsfile
4f56034 · 1 minute ago
History
Code
Blame
20 lines (20 loc) · 406 Bytes
Raw
pipeline {
agent any
stages {
stage('Build') {
steps {
echo 'Building the application...'
}
}
stage('Test') {
steps {
echo 'Testing the application...'
}
}
stage('Deploy') {
steps {


Jenkins / GitHub-Jenkins-DEMO / #4
+refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe config remote.origin.url https://github.com/Yogini21-jpg/jenkins-html-demo.git # timeout=10
> git.exe config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision 4f56034a007cd50959b8838d35d20b9a1b8ca0f4 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f 4f56034a007cd50959b8838d35d20b9a1b8ca0f4 # timeout=10
Commit message: "Update Jenkinsfile"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage

```
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Testing the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

CONCLUSION:

Implementing this CI/CD pipeline successfully transforms manual software delivery into a fully automated, event-driven workflow. By integrating a GitHub repository with Jenkins via webhooks, any new code modification instantly triggers a predefined sequence of operations. Utilizing a Declarative Pipeline defined in a Jenkinsfile ensures that the integration process is consistently executed across isolated Build, Test, and Deploy stages. This automated orchestration not only accelerates the release cycle by providing immediate feedback on code health but also establishes a reliable, scalable foundation for modern DevOps practices by completely removing manual intervention from the deployment lifecycle.